

Verifiable Solvency For Crypto Exchanges: A Zero-Knowledge Proof Approach

Vincent Xu - yx2021

CSCI-GA-3033-107

Cryptography and Blockchain

Professor: Benedict Bunz

May 2026

I. Introduction

In November 2022, FTX — at the time one of the largest cryptocurrency exchanges in the world — collapsed, revealing an \$8 billion gap between customer deposits and available assets. The episode exposed a structural problem with centralized exchanges: users must trust the exchange's word about whether their funds are safe. Even public balance-sheet disclosures are inadequate, because they can be fabricated, selectively timed, or limited to assets while concealing liabilities.

A natural response is to use cryptographic proofs. If an exchange can prove, without revealing sensitive data, that its total assets exceed its total liabilities, users have a verifiable guarantee rather than an unverifiable claim. Zero-knowledge (ZK) proofs make this possible: the exchange proves a statement is true without disclosing the private inputs that witness it.

However, a naive ZK solvency proof still admits a critical attack. An exchange can exclude users with large balances from the liability set, prove solvency over the reduced subset, and publish a valid proof. Because the proof makes no commitment to which users were included, a third party cannot distinguish this from an honest proof over the full user base. This is the omitted-liability attack, and it is the central problem this project addresses.

This project builds a ZK solvency pipeline on top of SP1, Succinct's zkVM, with four components working together to close this gap. First, a Rust zkVM program takes user balances and reserve balances as private inputs, builds a SHA-256 Merkle tree over all user balances inside the proof, asserts that total assets meet or exceed total liabilities, and emits a binding public output containing the Merkle root, an assets commitment, and both totals. Second, a proof generation script submits this program to the Succinct Prover Network and produces a 964-byte PLONK proof. Third, a Solidity smart contract verifies the proof on-chain via the SP1 PLONK gateway and stores the attested values immutably. Fourth, a Next.js web frontend lets any user verify, entirely in the browser using the Web Crypto API, that their account balance is included in the attested Merkle root.

Because the Merkle root is computed inside the ZK proof rather than supplied as an external input, the exchange cannot use a different tree when answering user queries — the root in the proof and the root used for inclusion checks are the same object. This design improves

verifiability in a meaningful way, though it does not eliminate all forms of exchange dishonesty: completeness of the liability set still depends on the exchange providing honest inputs, and reserve ownership is not proven on-chain.

The remaining parts of this report cover the threat model, background, system design, implementation, Sepolia deployment, evaluation, and security analysis for this prototype.

II. Problem Setting and Threat Model

2.1 Parties

Four principals interact in this system. The exchange holds user funds and constructs the proof. Users have balances held by the exchange and want to verify their funds are accounted for. The on-chain verifier is the SP1 PLONK gateway contract, which checks proof validity. Public observers, including auditors and other users, can read attested values from the smart contract without any special access.

2.2 Solvency Defined

Solvency is defined at a point in time: an exchange is solvent if the sum of all reserve balances (assets it controls) is greater than or equal to the sum of all user balances (liabilities it owes). Formally, $\text{total_assets} \geq \text{total_liabilities}$ at the snapshot moment. This prototype uses unsigned 64-bit integers for both quantities, expressed in the exchange's internal unit of account.

2.3 The Omitted-Liability Attack

Consider an exchange with 10,000 users, of which 100 hold very large balances. The exchange selects the 9,900 remaining users, generates a valid ZK proof that assets exceed the liabilities of this subset, and publishes the proof. The proof is cryptographically valid — the prover did not lie about the computation — but the claim is misleading. Because the proof commits only to the Merkle root of the selected users, not to the complete user set, a verifier cannot tell whether anyone was excluded.

An excluded user who attempts to verify their inclusion will find that no valid proof exists against the attested root. But this failure is ambiguous — it could reflect malicious exclusion, a data entry error, or a registration gap. More importantly, because the proof commits only to the Merkle root of *whatever inputs the exchange chose to provide*, no third party can demonstrate that the exchange omitted anyone. There is no published ground truth to compare the attested tree against.

2.4 What Users Need

For a solvency proof to be meaningful to individual users, two properties must hold. First, a user must be able to verify that their account is part of the liability sum the proof covers. Second, the Merkle commitment used for inclusion proofs must be the same commitment that bounds the liability sum in the proof — otherwise an exchange could maintain two trees, one used to generate the proof and a different one used to answer user queries.

2.5 The Binding Requirement

The solution is to compute the Merkle root inside the ZK proof execution and emit it as a public output. This means the root is a deterministic function of the private inputs that were hashed during proving. An exchange cannot produce a valid proof over user set A and then claim to users that the root corresponds to user set B — the root is cryptographically bound to the exact set of inputs that were processed. This binding property is the primary design motivation described in Section 4.

III. Background and Related Work

3.1 Background

Merkle Trees. A Merkle tree is a binary hash tree in which each leaf encodes one data item and each internal node is the hash of its two children. The root summarizes the entire dataset in a single 32-byte value. Given a root, any party can verify that a specific leaf is part of the committed dataset by checking a sibling path of $O(\log N)$ hashes, called an inclusion proof. This

makes Merkle trees well-suited for committing to large datasets while keeping per-user verification cheap.

Zero-Knowledge Proofs and SNARKs. A zero-knowledge proof (ZKP) is a protocol in which a prover convinces a verifier that a statement is true without revealing any information beyond the truth of the statement. In practice, solvency proofs use succinct non-interactive arguments of knowledge (SNARKs), which produce a short, constant-size proof that a verifier can check efficiently. The key properties are soundness (a dishonest prover cannot produce a valid proof for a false statement, except with negligible probability) and zero-knowledge (the proof reveals nothing about the private witness). The public inputs and outputs of the computation are visible to the verifier; the private inputs are not.

SP1 zkVM. SP1 is a zero-knowledge virtual machine developed by Succinct that executes Rust programs compiled to a RISC-V ELF binary and produces a ZK proof of correct execution. The developer writes an ordinary Rust program; SP1 handles circuit generation. SP1 v6.1.0 supports two proof systems: PLONK (on the GnarK BN254 curve) and Groth16. This project uses PLONK. The choice was not aesthetic: SP1 v6.1.0's on-chain Groth16 verifier had a known circuit bug that made Groth16 proofs unsafe for on-chain submission at the time of development. PLONK was the recommended alternative, and the SP1 PLONK gateway on Sepolia (0xd685a80aF2d1761648e56716af4868d850Dae49B) was available and functional. PLONK proofs are constant-size (964 bytes in this system) regardless of program complexity, which makes on-chain verification cost independent of the number of users.

3.2 Related Work

The idea of using Merkle trees for proof of reserves originates with Greg Maxwell's 2013 proposal, which described how an exchange could commit to user balances via a Merkle tree and allow each user to verify their leaf was included. This approach does not use zero-knowledge proofs: the auditor sees the full tree, which leaks information about other users' balances, and the exchange's total assets are asserted rather than proven cryptographically.

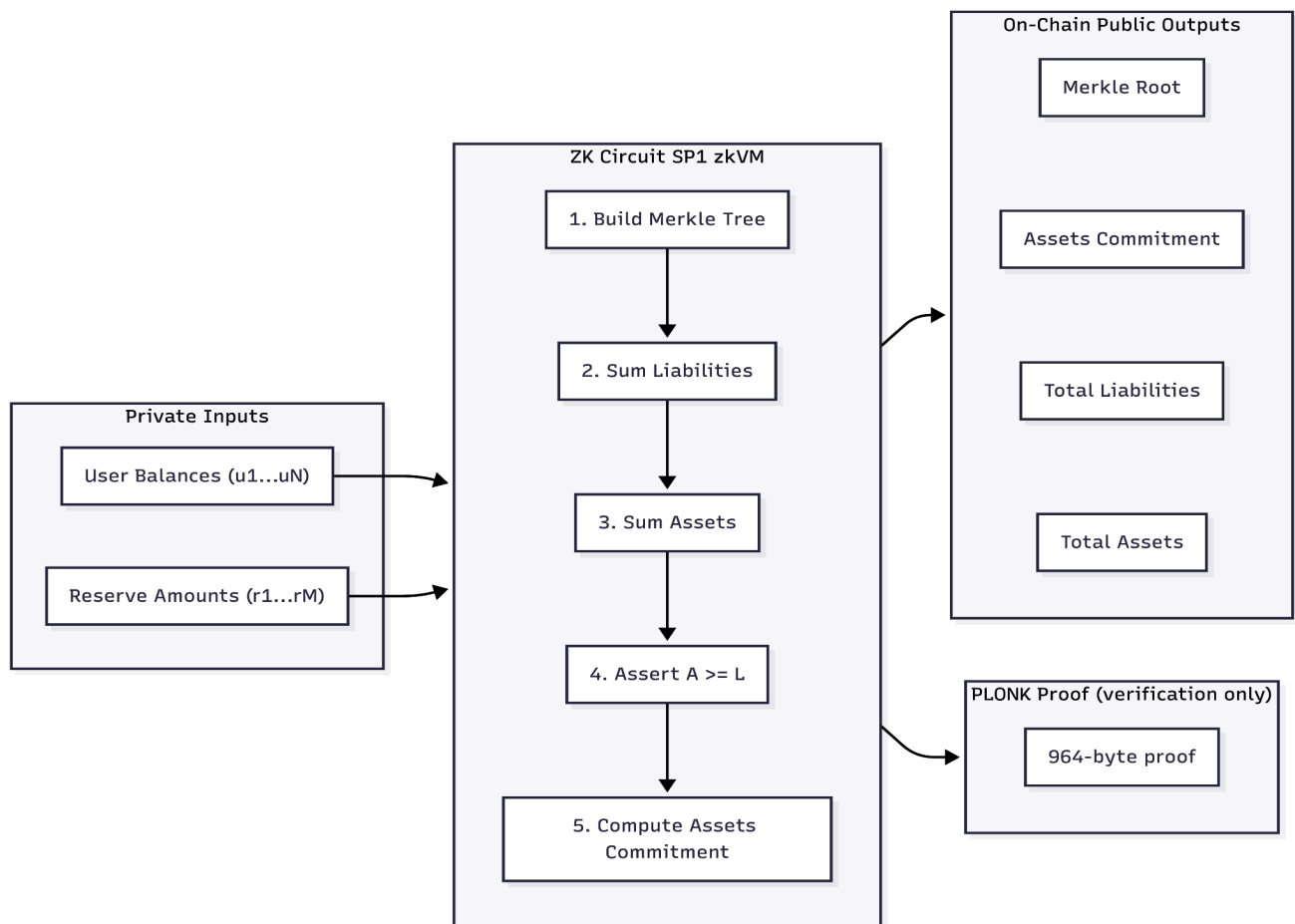
More recently, major exchanges including Binance and Kraken have published audit-based proofs of reserves using Merkle commitments. These proofs are prepared with the help of

external auditors who inspect the underlying data. While this reduces the trust surface somewhat, it does not eliminate it: users must trust that the auditor correctly verified the inputs and was not compromised or misled. There is no cryptographic guarantee that the auditor-checked data matches the on-chain commitment.

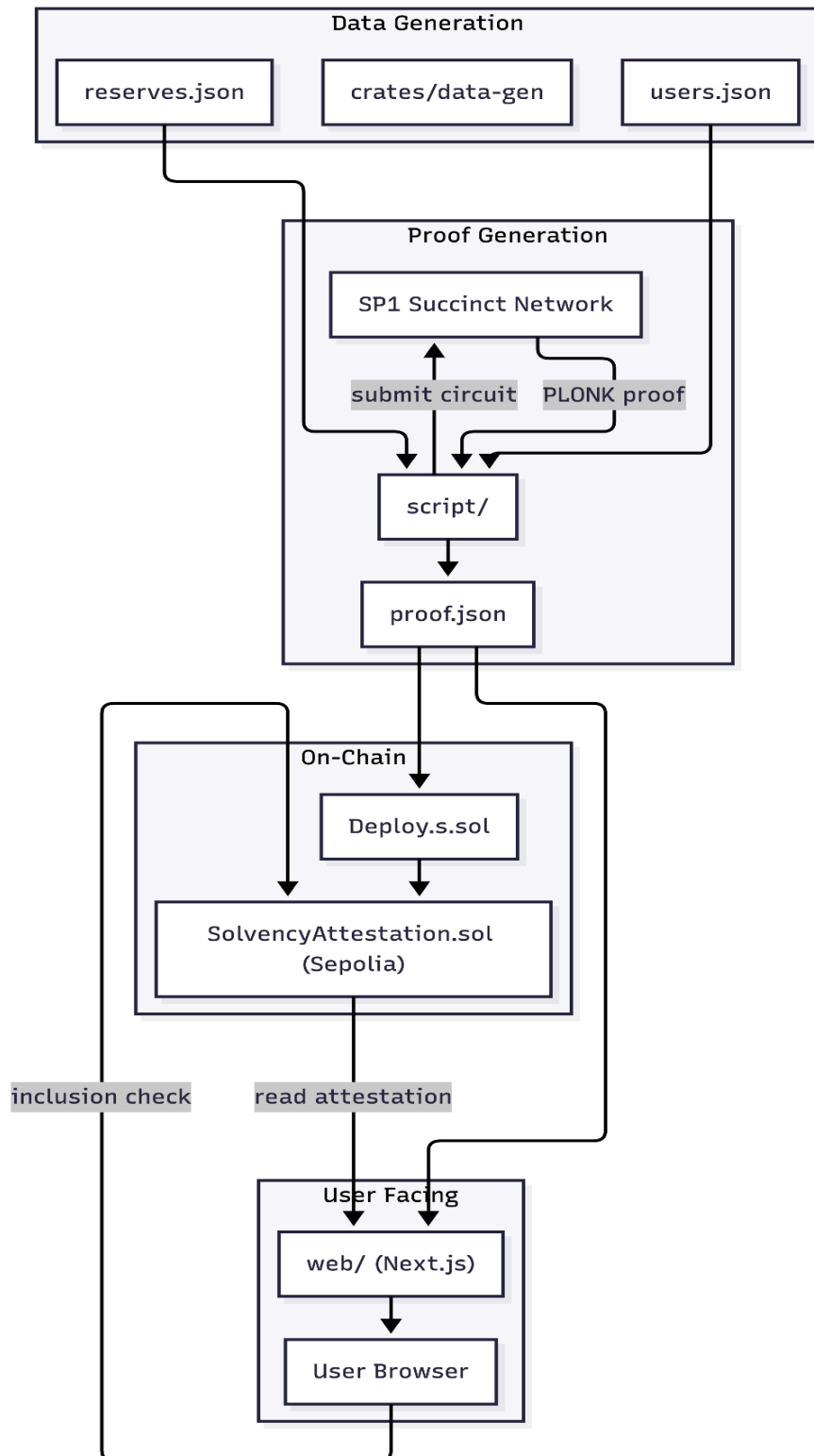
This prototype takes a different approach: the Merkle root and the solvency assertion are produced by a single ZK computation, with no trusted intermediary. Any user can verify the proof by checking the on-chain transaction, and any user can verify their own inclusion in the browser without downloading or running any native software.

IV. System Design

4.1 System Workflow



4.2 Architecture Overview



The workflow has three logical layers. In the proving layer, the exchange provides two lists of private inputs to the SP1 zkVM program: a list of user balances (liabilities) and a list of reserve balances (assets). The program executes entirely inside the zkVM, producing a PLONK proof and a 128-byte public output. In the on-chain layer, the proof and public output are submitted to `SolvencyAttestation.sol`, which delegates proof verification to the SP1 PLONK gateway and stores the attested values. In the client layer, any user can read the attested Merkle root from the contract (or from the `proof.json` artifact) and verify their own inclusion: the browser fetches the sibling path from the server, then independently recomputes the leaf hash from the user-supplied ID and balance using the Web Crypto API and verifies the path against the on-chain root. The critical computation — deriving the leaf hash — runs entirely client-side, so the server cannot misrepresent a user's balance without causing verification to fail.

4.3 Privacy Boundary

Private inputs, known only to the exchange and the zkVM prover, are the list of user balances (each a (id, balance) pair) and the list of reserve balances (each also (id, balance)). No individual balance or identity is revealed by the proof.

Public outputs, visible to anyone, are the `merkleRoot` (bytes32), the `assetsCommitment` (bytes32), `totalLiabilities` (uint64), and `totalAssets` (uint64), encoded together as a 128-byte ABI-encoded tuple. From these four values a verifier can confirm that assets exceed liabilities and can check individual inclusion proofs against the committed root, without learning any individual balance.

4.4 The Binding Argument

The central design property is that the Merkle root is computed inside the zkVM execution, not supplied as an external input. In the guest program (`crates/program/src/main.rs`), the very first operation on user data is `MerkleTree::build(&users)`, which hashes every leaf and constructs the tree. The resulting root is captured and later committed as part of the public output via `sp1_zkvm::io::commit_slice`. Because SP1's soundness guarantee covers the entire execution trace, a cheating prover cannot substitute a different root without invalidating the proof.

This rules out the split-tree attack: the exchange cannot run the solvency assertion over user set A (to make the arithmetic look good) while advertising a Merkle root derived from user set B (to let users find themselves). Both the root and the solvency totals come from the same execution over the same input data.

4.5 Public Output Format

The 128-byte public output is Solidity ABI-encoded as a tuple (bytes32, bytes32, uint64, uint64):

- Bytes 0-31: `merkleRoot` — SHA-256 Merkle root over all user balance leaves
- Bytes 32-63: `assetsCommitment` — SHA-256 hash of all reserve (id, balance) pairs
- Bytes 64-95: `totalLiabilities` — sum of all user balances (uint64 in a 32-byte slot)
- Bytes 96-127: `totalAssets` — sum of all reserve balances (uint64 in a 32-byte slot)

The encoding is handled by the `alloy_sol_types` crate inside the zkVM program, ensuring that the on-chain decoder (`abi.decode` in Solidity) reads the same layout.

4.6 Key Design Decisions

SHA-256 over Keccak. SP1 includes a SHA-256 precompile that accelerates SHA-256 hashing inside the zkVM at the circuit level. Keccak (Ethereum's native hash function) has no equivalent precompile in SP1 v6.1.0, making it substantially more expensive in terms of RISC-V cycle count. Since the Merkle tree construction dominates proof generation time for large N, SHA-256 was the practical choice. The TypeScript frontend mirrors this by using the Web Crypto API's SHA-256 implementation, ensuring hash compatibility.

Assets commitment design. The `assetsCommitment` is a single SHA-256 hash computed by iterating over all reserves and feeding each reserve's id bytes followed by its balance bytes into the hasher (`assets_commitment` in `crates/types/src/lib.rs`). Including the reserve ID in the hash means two reserve sets with the same total but different composition or ordering produce different commitments, preventing an exchange from substituting one reserve configuration for another without detection.

Padding strategy. The Merkle tree pads the leaf list to the next power of two by repeating the last leaf. This is simpler and cheaper inside the zkVM than zero-padding (which would require a sentinel value distinguishable from real leaves) or balanced-but-non-power-of-two trees. The TypeScript implementation in `web/_lib/merkle.ts` explicitly mirrors this strategy; any deviation would cause the browser-computed root to disagree with the on-chain root, silently breaking inclusion verification.

Separate script workspace. The proof generation script lives in `script/` with its own `Cargo.toml` rather than as a member of the root workspace. This isolates it from a `serde_core` version conflict between `sp1-sdk` and `alloy` (used by the `types` crate), which is a known upstream compatibility issue. The `types` crate is shared between the script and the guest via a path dependency that resolves correctly in both contexts.

V. Implementation

5.1 Shared Types and Hashing (`crates/types`, `crates/data-gen`)

The `types` crate defines the two core data structures: `UserBalance { id: u64, balance: u64 }` and `ReserveBalance { id: u64, balance: u64 }`, both serializable with `serde`. It also provides the `MerkleTree` implementation and the `assets_commitment` function used by both the zkVM guest and the inclusion proof CLI.

`MerkleTree::build` takes a slice of `UserBalance` values. It hashes each leaf as `SHA-256(id_le64 || balance_le64)` — eight bytes for the ID in little-endian order followed by eight bytes for the balance — to produce a 32-byte leaf hash. It then pads the leaf array to the next power of two by repeating the last leaf, and builds the tree bottom-up, hashing pairs as `SHA-256(left_32bytes || right_32bytes)`. The resulting tree stores all layers, which are needed for inclusion proof generation.

`MerkleTree::prove(index)` extracts the sibling path for a given leaf index, returning a `MerkleProof` containing the sibling hashes and path-bit flags. `MerkleProof::verify(leaf, expected_root)` recomputes the root from a leaf hash and sibling path and checks it against the expected root — the same logic is mirrored in `web/_lib/merkle.ts` for browser-side verification.

The data-gen crate provides a CLI binary that generates synthetic user and reserve data with configurable size and surplus ratio, writing JSON files to data/users.json and data/reserves.json. It guarantees solvency by construction: it first generates user balances, computes the total, and then generates reserve balances such that their sum is $(1 + \text{surplus_ratio})$ times the total liabilities.

5.2 zkVM Guest Program (crates/program)

The guest program (crates/program/src/main.rs) is the computation that is proven. It reads private inputs using SP1's stdin interface — first the user balance list, then the reserve balance list — and performs five steps.

First, it calls `MerkleTree::build(&users)` to construct the SHA-256 Merkle tree over all users. Second, it sums user balances to obtain `total_liabilities` and sums reserve balances to obtain `total_assets`. Third, it asserts `total_assets >= total_liabilities`, panicking with "insolvent" if the assertion fails (which causes proof generation to fail). Fourth, it calls `assets_commitment(&reserves)` to compute the SHA-256 commitment over all reserves. Fifth, it ABI-encodes the four public values and commits them via `sp1_zkvm::io::commit_slice`.

This program is compiled to a RISC-V ELF binary using SP1's toolchain (`cargo prove build`) and embedded in the script binary at compile time via `include_bytes!`. The program verification key is derived live from the proving key using `pk.verify_key().bytes32()` — it is never hardcoded, so rebuilding the ELF automatically updates the vkey everywhere.

5.3 Proof Generation Pipeline (script/)

The script binary (script/src/main.rs) loads user and reserve data from JSON files, initializes a `ProverClient` using `ProverClient::from_env()`, and calls `client.setup()` to obtain a proving key and derive the program vkey. It writes private inputs to an `SP1Stdin` buffer and calls `client.prove(&pk, stdin).plonk().await` to request a PLONK proof.

The `SP1_PROVER` environment variable controls the backend. Setting `SP1_PROVER=mock` runs the program logic through SP1's execution engine without generating a real ZK proof — this is fast (sub-second for small N , ~27 seconds for $N=5000$) and useful for development.

Setting `SP1_PROVER=network` submits the job to the Succinct Prover Network, which generates a real PLONK proof in approximately 5-15 minutes. Mock proofs are written to `proof.mock.json` (gitignored); network proofs are written to `proof.json` (committed). The two output files are kept separate so that a development run cannot overwrite a real proof artifact.

5.4 Smart Contract (`contracts/src/SolvencyAttestation.sol`)

`SolvencyAttestation.sol` is a minimal Solidity contract (Solidity ^0.8.20) with two immutable fields set at construction: the address of the SP1 PLONK verifier gateway and the program verification key. Its single state-mutating function, `submitProof(bytes proofBytes, bytes publicValues)`, calls `VERIFIER.verifyProof(PROGRAM_VKEY, publicValues, proofBytes)` on the gateway. If the call reverts, the transaction fails; if it succeeds, the contract decodes the public values, stores them as an Attestation struct (`merkleRoot`, `assetsCommitment`, `totalLiabilities`, `totalAssets`, `timestamp`), and emits a `SolvencyProven` event.

The contract stores only the most recent attestation in its state; all prior attestations remain permanently accessible via the on-chain event history, since a `SolvencyProven` event is emitted and recorded on-chain with each successful proof submission. The Forge test suite covers 7 tests: 4 happy-path cases (valid proof accepted, values stored correctly, event emitted, attestation readable after submission) and 3 negative cases (wrong vkey rejected, tampered public values rejected, empty proof rejected).

5.5 Inclusion Proof CLI and Browser Verifier (`crates/inclusion`, `web/_lib/merkle.ts`)

The inclusion CLI (`crates/inclusion/src/main.rs`) takes a user ID as input, rebuilds the Merkle tree from the same `data/users.json` file used during proving, generates the sibling-path proof for that user, and verifies it against the Merkle root extracted from `proof.json`. Because the CLI uses the same `MerkleTree::build` call as the guest program, its recomputed root always matches the attested root — any discrepancy would indicate data tampering.

The TypeScript mirror (`web/_lib/merkle.ts`) reimplements the same logic using the Web Crypto API: `leafHash` computes `SHA-256(id_le64 || balance_le64)` via `crypto.subtle.digest`, `nodeHash`

computes $\text{SHA-256}(\text{left} \parallel \text{right})$, and `buildTree` pads to the next power of two by repeating the last leaf — exactly mirroring the Rust implementation. The browser's `InclusionChecker` component recomputes the user's leaf hash client-side from the user-supplied (id, balance) pair, then verifies it against the sibling path fetched from the server and the Merkle root read from on-chain. Because the leaf hash is derived independently in the browser, a malicious server cannot misrepresent a user's balance without causing verification to fail — though it can still withhold or corrupt the sibling path, producing a failure the user cannot easily diagnose.

5.6 Frontend (web/)

The `Next.js` frontend reads `proof.json` and `deployment.json` at build time (server component) and renders an attestation card showing the on-chain contract address, proof transaction hash, Merkle root, and solvency totals, with a direct Etherscan link. The client-side `InclusionChecker` component accepts a user ID and balance, computes the leaf hash in the browser, fetches the sibling path, and verifies inclusion, reporting success or failure without contacting any backend.

VI. Sepolia Deployment

The full pipeline was executed end-to-end on the Ethereum Sepolia testnet, demonstrating that all components work together in practice.

The contract `SolvencyAttestation` was deployed at address `0x97d55Ff73f7592F85AafF025a94963d02266cC78`, configured with the SP1 PLONK gateway at `0xd685a80aF2d1761648e56716af4868d850Dae49B` and the program verification key `0x0098ee1f091411258d9318cb9a146c4e48145cee16b45a774d0445772cbfca4f`.

The dataset used for the proof was generated by the data-gen tool: 100 users and 5 reserve accounts. The attested public values are a `totalLiabilities` of 501,258, a `totalAssets` of 601,509, and a surplus of +100,251 (approximately 20% over the liability total). The Merkle root committed in the proof is

`0xc62b97ef52f4d1c1139f3d829235bfa7510b43beb1da0bf0d1b2f961452bb41b`, and the assets commitment is `0xe9f2b15ad68b93aeb0dca75ab6fb017446e547ad0b2725d78405d5dbefb2f23a`.

The PLONK proof was generated via the Succinct Prover Network and is 964 bytes. The first four bytes (0x5a093a2f) are the SP1 v6.1.0 PLONK verifier selector, used by the gateway to route the verification call to the correct versioned verifier contract (0xc3c6dDDAc8829b233Dc6536Ec024775a57b0AF2A). The proof was submitted on-chain in transaction 0xb952c483e839b5cfbe3694ac4e3a3ace9a643d2dea2273d867dd5c5ea8f43ea3 at block 10746772. The transaction succeeded, the SP1 PLONK gateway accepted the proof, and the SolvencyProven event was emitted. The attested values are now permanently stored in the contract's latestAttestation field and readable by anyone.

The web frontend was verified against this deployment: the attestation card displayed the correct on-chain values, and the InclusionChecker successfully verified inclusion for a sample user ID, with all Merkle path computation running in the browser using the Web Crypto API.

VII. Evaluation

7.1 Merkle Tree Scaling

Merkle tree operations are benchmarked in native Rust (release build). Build time is $O(N)$, dominated by the N SHA-256 leaf hashes: at $N=100$ it is 0.088 ms, at $N=1000$ it is 0.458 ms, and at $N=5000$ it is 3.116 ms. Prove and verify times are $O(\log N)$ and are effectively constant across all tested sizes: prove ranges from 0.121 us to 0.167 us, and verify ranges from 2.575 us to 3.780 us. The depth of the tree after padding is $\text{ceiling}(\log_2(N))$; for $N=100$ this is 7 levels (128 padded leaves), meaning an inclusion proof consists of exactly 7 sibling hashes.

Table 1: Merkle tree operation times by N

N (users)	Build (ms)	Prove (us)	Verify (us)
10	0.016	0.121	2.575
100	0.088	0.137	3.124
500	0.271	0.167	3.073
1,000	0.458	0.142	2.905

5,000	3.116	0.149	3.780
-------	-------	-------	-------

7.2 zkVM Proof Generation Scaling

SP1 mock mode times the full zkVM execution — program logic, SHA-256 precompile calls, and public output commitment — without generating a real ZK proof. These times grow roughly linearly with N, reaching approximately 27 seconds at N=5000. It is important to note that mock execution time measures the trace generation cost, not cryptographic proving time. Real PLONK proof generation via the Succinct Prover Network took approximately 5-15 minutes for the N=100 dataset, and real proving time does not scale simply with mock time. The mock numbers are useful for comparing relative cost across dataset sizes, not for estimating production proving latency.

Table 2: SP1 mock execution time by N

N (users)	SP1 Mock Time (s)
10	0.074
100	0.469
500	1.947
1,000	4.031
5,000	27.152

7.3 On-Chain Cost and Artifact Sizes

The on-chain gas cost of submitProof is approximately 250,000 gas regardless of N. This is because the SP1 PLONK gateway verifies a fixed-size proof (the same 964-byte proof regardless of the number of users), and Solidity ABI decoding of the 128-byte public output is a constant-cost operation. The cost does not grow with the size of the user dataset.

Both the proof and the public values are constant in size: the PLONK proof is always 964 bytes, and the ABI-encoded public output is always 128 bytes (four 32-byte slots). This means the on-chain footprint of a solvency attestation is the same whether the exchange has 100 users or 1,000,000 users.

The asymmetric cost structure is notable from a systems perspective: the exchange bears $O(N)$ cost (constructing the Merkle tree during proof generation), while each user bears only $O(\log N)$ cost to verify their inclusion, and the verifier bears $O(1)$ cost for on-chain proof verification.

VIII. Security Analysis and Limitations

8.1 Security Guarantees

When a proof verifies successfully on-chain, the following statements hold with cryptographic confidence, assuming SP1's soundness:

The exchange's total assets, as committed in the proof, are at least as large as the total liabilities committed in the same proof. The Merkle root committed in the proof was computed by the SP1 guest program from the same user balance list that was summed to produce `totalLiabilities` — these two quantities are bound together by the proof. The on-chain attestation, stored in the contract's `latestAttestation` field, is immutable and publicly readable; it cannot be altered retroactively.

Any user who can produce a valid Merkle inclusion proof against the attested root can be certain their balance is part of the liability sum that the exchange proved solvency against.

8.2 Trust Assumptions

Despite these guarantees, several trust assumptions remain. The exchange must provide a complete and accurate list of user balances as private inputs. The ZK proof binds the root to the inputs provided, but it cannot verify that all users were included — it only proves that the computation was performed correctly on whatever inputs were given. This is the residual omitted-liability risk discussed below.

The security of the proof itself relies on SP1's soundness, which in turn relies on the security of the PLONK argument system and the BN254 pairing assumption. SHA-256 collision resistance is assumed for both the Merkle tree and the assets commitment. Finally, the Rust implementation and the TypeScript implementation of the Merkle hash functions must agree byte-for-byte: leaf encoding uses little-endian 64-bit integers (`id_le64 || balance_le64`), and any discrepancy between the two implementations would silently produce a wrong root, causing all browser inclusion checks to fail.

8.3 Limitations

The omitted-liability attack is mitigated but not eliminated. This design ensures that if a user's inclusion proof verifies, their balance is genuinely part of the proven liability sum. However, a user whose ID does not appear in the attested tree has no cryptographic way to determine whether they were excluded maliciously or simply not yet registered. The proof provides no mechanism for an excluded user to raise a complaint.

Reserve ownership is not proven. The assets commitment cryptographically binds the exchange to a specific set of reserve (id, balance) pairs, but it does not prove that the exchange controls those reserves on-chain. Reserve balances are private inputs, not verified against any on-chain state. A more complete system would include on-chain balance attestations or cryptographic signatures from reserve addresses.

The proof is a snapshot. Solvency is asserted at a single point in time, and there is no cryptographic freshness guarantee. An exchange could generate one honest proof and then immediately withdraw reserves. Adding block-height commitments or periodic attestation requirements would improve freshness guarantees but would require changes to the contract and the proving pipeline.

User identity is opaque. User IDs are arbitrary integers assigned by the exchange. A user must trust the exchange to assign them the correct ID; they cannot independently derive their ID from publicly known information. A future improvement (discussed in Section 9) is to derive IDs from a hash of the user's email address or public key, so users can compute their own ID without trusting the exchange.

Browser sibling-path delivery depends on TLS. The browser inclusion checker fetches the sibling path from the server. If the server delivers a forged path (while the root is correctly read from on-chain), the verification will fail, but a user who receives a failure might not know whether they were actually excluded or whether the server sent bad data. Reading the sibling path from a decentralized source would remove this dependency.

8.4 Implementation Correctness Note

The tree padding and leaf encoding must match exactly between `crates/types/src/merkle.rs` and `web/_lib/merkle.ts`. Both implementations pad to the next power of two by repeating the last leaf hash (not by appending zeros), and both compute leaf hashes as `SHA-256(id_le64 || balance_le64)` using little-endian byte order. A mismatch in either convention silently produces a different root and breaks all inclusion proofs without an obvious error message.

IX. Future Work

Several extensions would meaningfully strengthen this prototype.

User identity binding. The current design uses opaque integer IDs assigned by the exchange. A user must trust the exchange's ID assignment to know which leaf corresponds to their account. A natural improvement is to derive user IDs as $H(\text{email})$ — a hash of the user's verified email address — so that any user can independently compute their own ID and look up their leaf without trusting the exchange. This would require a registration mechanism (e.g., the exchange publishing a signed mapping from email hash to leaf index) but would eliminate one trust assumption from the inclusion-checking flow.

Reserve privacy via a second Merkle tree. The current `assetsCommitment` reveals nothing about individual reserves to outside observers, but the total asset figure (`totalAssets`) is public. An exchange that wants to keep its reserve allocation private while still proving total assets exceed total liabilities could commit reserves in a second Merkle tree, prove the sum inside the zkVM, and publish only the tree root and total — mirroring the treatment of user balances. This would

allow proving asset totals without revealing how many reserves exist or what individual reserve balances are.

Scalable Merkle path storage. The current inclusion proof CLI rebuilds the full Merkle tree on every query, which requires reading all N user records. For exchanges with millions of users this is impractical. Pre-computing and storing sibling paths at attestation time, delivering user-specific paths as part of the attestation, or using sparse Merkle trees (which support non-membership proofs and compressed storage) are all viable directions depending on the deployment requirements.

X. Conclusion

This project demonstrates a complete, end-to-end ZK solvency pipeline: a Rust zkVM program compiled to RISC-V, proved via the Succinct Prover Network, verified by a Solidity smart contract deployed on Ethereum Sepolia, and surfaced to users through a browser-based inclusion checker.

The central contribution of the design is the binding of the Merkle root and the solvency assertion within a single ZK computation. Because the Merkle root is emitted as a public output of the proved program rather than provided as an external input, the exchange cannot use a different user set when answering individual inclusion queries. This closes the omitted-liability gap that exists in naive ZK solvency constructions.

The pipeline is practical today. SP1 and the Succinct Prover Network abstract away circuit development; Foundry handles smart contract deployment and testing; the Web Crypto API brings browser-native hash computation that matches the Rust implementation. The on-chain cost is approximately 250,000 gas per attestation, constant regardless of user count, and the proof artifact is 964 bytes.

What remains outside the guarantee is equally important to state clearly. The proof does not verify that all users were included in the liability set — that depends on honest input from the exchange. The proof does not verify that the exchange controls the assets it claims. And the

proof is a snapshot, not a continuous commitment. These limitations define the boundary of what this prototype improves and what future work would be needed to address.